

【rsync 原理】差异同步算法：滚动校验与两级匹配

2026-06-29 · linux · network · #rsync #rolling-checksum #delta-encoding #adler #xxhash
#file-sync

文章导航

分类入口

linux

network

标签入口

#rsync

#rolling-checksum

#delta-encoding

#adler

#xxhash

#file-sync

把一个 1GB 的文件改掉中间一个字节，再 `rsync` 到远端，传输量只有几十 KB，而不是 1GB。这件事看起来理所当然，但只要追问一句就会卡住：源端（**sender**）手里只有新文件，它根本看不到远端那份旧文件的内容，凭什么知道哪几个字节变了？

更麻烦的是，改动往往不是“原地替换”，而是插入或删除。一旦在文件开头插入一个字节，后面所有内容的字节偏移整体平移。任何“按固定位置逐块比较”的方案都会从插入点开始全部失配，退化成传整个文件。`rsync` 要解决的恰恰是这种未对齐的差异。

Andrew Tridgell 和 Paul Mackerras 在 1996 年的技术报告《The rsync algorithm》(TR-CS-96-05) 给出的答案是一套两级校验匹配：远端把旧文件切块、算出每块的校验和发给源端；源端拿着新文件，用一个能 $O(1)$ 滑动的弱校验在每一个字节偏移上试探，命中后再用强校验确认，从而在不知道旧文件内容的情况下，找出新文件里哪些片段能直接复用旧块、哪些必须当作新数据传过去。

这篇只讲“差异是怎么算出来的”这一层算法，所有结论对照 `rsync 3.4.4` 源码

(`checksum.c`、`match.c`、`generator.c`)。至于这套算法在真实 rsync 里由哪些进程跑、wire protocol 长什么样、实测能省多少，放在 [协议](#)、[进程模型与实测](#) 那篇。

一、把问题写清楚

约定两台机器、两份文件：

- 接收端 (receiver) 有一份旧文件，记作基准文件 (basis file) B 。
- 发送端 (sender) 有一份新文件 A ，要让接收端最终得到 A 。

目标是让 A 落到接收端，同时网络上传输的字节数尽量接近 A 与 B 的真实差异，而不是 A 的全长。约束是：发送端不能读接收端的 B ，接收端也不能读发送端的 A ，双方只能通过一条管道交换少量信息。

rsync 的总体思路分三步：

1. 接收端把 B 切成固定长度 L 的块，对每块算一对校验和（一个弱的、一个强的），把这些校验和发给发送端。这套校验和称为签名 (signature)。
2. 发送端拿着签名扫描 A ：在 A 的每个字节偏移上，判断“从这里开始的 L 个字节”是不是等于 B 里的某个块。能匹配上的，记成一个指向块号的引用；匹配不上的字节，作为字面数据 (literal)。
3. 发送端把“引用 + 字面数据”交织成的指令流发给接收端，接收端照着它，从 B 里复制能复用的块、插入收到的字面数据，拼出 A 。

第 1、3 步是工程，难点全在第 2 步：发送端要在 A 上逐字节地问“这里是不是某个旧块”。一个文件几亿个偏移，每个偏移都算一次强校验显然不可行。rsync 的设计核心，就是把这个高频试探做到极快。

二、弱滚动校验：让逐字节试探变成常数操作

逐字节试探之所以可行，靠的是一个能 $O(1)$ 增量更新的弱校验和。技术报告里把它定义成两部分。对块 $X_k, X_{k+1}, \dots, X_l$ （共 $L = l - k + 1$ 个字节）：

$$a(k, l) = \left(\sum_{i=k}^l X_i \right) \bmod M, \quad b(k, l) = \left(\sum_{i=k}^l (l - i + 1) X_i \right) \bmod M$$
$$s(k, l) = a(k, l) + M \cdot b(k, l), \quad M = 2^{16}$$

a 是块内字节的简单和， b 是带权和（越靠左权重越大）。两个 16 位的量拼成一个 32 位校验和 s 。这个构造受 Mark Adler 的 Adler-32 启发，源码注释也是这么写的。

`checksum.c` 里 `get_checksum1()` 计算它（这里取 `CHAR_OFFSET == 0`，即 `rsync.h` 的默认值，省去常数项）：

```
/* checksum.c, rsync 3.4.4: CHAR_OFFSET 默认为 0，下面按 0 理解 */
uint32 get_checksum1(char *buf1, int32 len)
{
    int32 i;
    uint32 s1, s2;
    schar *buf = (schar *)buf1;

    s1 = s2 = 0;
    for (i = 0; i < (len-4); i+=4) {
        s2 += 4*(s1 + buf[i]) + 3*buf[i+1] + 2*buf[i+2] + buf[i+3];
        s1 += (buf[i+0] + buf[i+1] + buf[i+2] + buf[i+3]);
    }
    for (; i < len; i++) {
        s1 += buf[i]; s2 += s1;
    }
    return (s1 & 0xffff) + (s2 << 16);
}
```

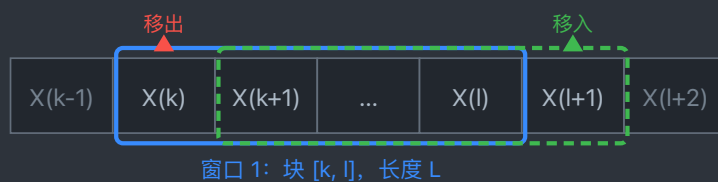
`s1` 对应 a ，`s2` 对应 b ，最后 `(s1 & 0xffff) + (s2 << 16)` 就是 $s = a + 2^{16}b$ 。循环按 4 字节展开只是为了快，算出来的还是上面那两个和。

关键性质在“滑动”。当窗口从 $[k, l]$ 右移一格到 $[k+1, l+1]$ 时，去掉最左的 X_k 、加入最右的 X_{l+1} ，新校验和不必重扫整块，只要：

$$a(k+1, l+1) = (a(k, l) - X_k + X_{l+1}) \bmod M$$

$$b(k+1, l+1) = (b(k, l) - L \cdot X_k + a(k+1, l+1)) \bmod M$$

弱滚动校验的 $O(1)$ 滑窗更新



窗口右移一字节（去掉 $X(k)$ ，加入 $X(l+1)$ ）：

$$\begin{aligned} a' &= a - X(k) + X(l+1) \\ b' &= b - L * X(k) + a' \\ s' &= a' + 65536 * b' \quad (M = 2^{16}) \end{aligned}$$

每滑动一格只做常数加减，无需重扫整块；这是 rsync 能在源端逐字节试探的前提。

弱滚动校验在窗口右移一字节时只做常数加减更新 a 与 b

这正是 `match.c` 的 `hash_search()` 里那段更新（`map[0]` 是要移出的字节，`map[k]` 是要移入的字节，`k` 是块长 L ）：

```
/* match.c, rsync 3.4.4, hash_search() 内, CHAR_OFFSET 当作 0 */
/* Trim off the first byte from the checksum */
s1 -= map[0];
s2 -= k * map[0];

/* Add on the next byte (if there is one) to the checksum */
if (more) {
    s1 += map[k];
    s2 += s1;
} else
    --k;
```

有了 $O(1)$ 更新，发送端就能从偏移 0 开始，一个字节一个字节地往后挪，每挪一格只做几次加减，立刻得到当前窗口的弱校验和。整趟扫描是 $O(|A|)$ 的。这一步把“在每个偏移试探”从天方夜谭变成了线性扫描。

弱校验便宜，代价是不可靠：32 位、线性结构，不同内容很容易撞出相同的 s 。所以它只能用来快速排除绝大多数不可能匹配的偏移，不能用来下匹配的结论。下结论是强校验的事。

三、两级匹配：弱校验筛、强校验定

发送端收到的签名里，每个旧块带两样东西：弱校验和 `sum1`（上面的 `s`）和一段强校验 `sum2`。匹配按两级走，先用弱校验快速筛，再用强校验确认。

第一级是哈希查找。发送端把所有旧块按弱校验和建一张哈希表。块数不多时用传统的 16 位表，哈希函数把 32 位的 `sum1` 折叠成 16 位桶号；块数很多时改用一张按 80% 装载率动态定大小的表。`match.c`：

```
/* match.c, rsync 3.4.4 */
#define SUM2HASH2(s1,s2) (((s1) + (s2)) & 0xFFFF)
#define SUM2HASH(sum) SUM2HASH2((sum)&0xFFFF, (sum)>>16)
#define BIG_SUM2HASH(sum) ((sum)%tablesize)

static void build_hash_table(struct sum_struct *s)
{
    /* 让大文件的哈希装载率约 80%；大于传统表时取奇数，保证 s2 能覆盖整个区间 */
    tablesize = (uint32)(s->count/8) * 10 + 11;
    if (tablesize < TRADITIONAL_TABLESIZE) /* 1<<16 */
        tablesize = TRADITIONAL_TABLESIZE;
    /* ...分配并用 0xFF 清空 (-1 表示空桶) ... */
    /* 同桶的块用 s->sums[i].chain 串成链表 */
}
```

扫描时，发送端先算当前偏移的弱校验和落在哪个桶。桶空就直接跳过——这是最常见的情况，也是弱校验存在的意义：用一次哈希查表就否决掉绝大多数偏移。桶非空，就顺着链表逐个比：

```
/* match.c, hash_search() 内层循环（节选） */
if (sum != s->sums[i].sum1)
    continue; /* 完整 32 位弱校验不等，跳过 */
l = (int32)MIN((OFF_T)s->blength, len-offset);
if (l != s->sums[i].len)
    continue; /* 块长不一致，跳过 */

if (!done_csum2) { /* 弱校验过关，才值得算强校验 */
    map = (schar *)map_ptr(buf, offset, l);
    get_checksum2((char *)map, l, sum2);
    done_csum2 = 1;
}

if (memcmp(sum2, sum2_at(s, i), s->s2length) != 0) {
    false_alarms++; /* 弱校验撞上、强校验否决 */
    continue;
}
```

两件事值得注意。第一，强校验 `get_checksum2()` 只在弱校验全匹配后才计算，而且对同一个偏移只算一次（`done_csum2`），所以昂贵的强校验调用次数被压到极低。第二，弱校验撞上、强校验否决的情况会被单独计数为 `false_alarms`（假阳）——这是弱校验不可靠的直接体现，调试输出里能看到它。

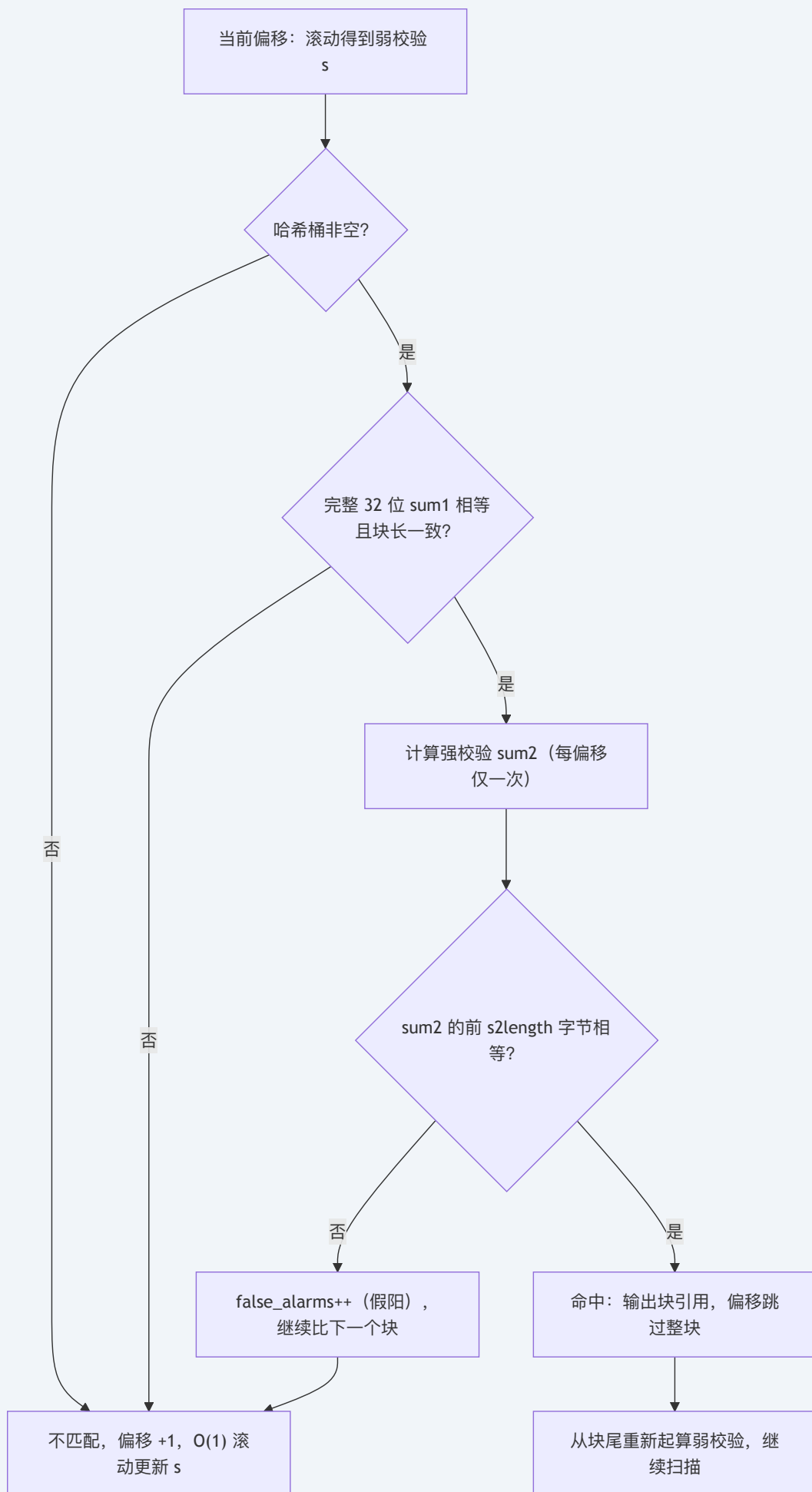
强校验 `get_checksum2()` 用的算法不是写死的 MD4/MD5，而是双方协商出来的传输校验 `xfer_sum_nni`：

```
/* checksum.c, get_checksum2() 按协商类型分派（节选） */
switch (xfer_sum_nni->num) {
case CSUM_XXH3_128: { /* xxh128 */
    XXH128_hash_t digest = XXH3_128bits_withSeed(buf, len, checksum_seed);
    SIVAL64(sum, 0, digest.low64);
    SIVAL64(sum, 8, digest.high64);
    break;
}
case CSUM_MD5: { /* ... md5_update / md5_result ... */ }
case CSUM_MD4: { /* ... mdfour ... */ }
}
```

两端都是 3.2 以后的版本时，会协商到双方都支持的最强算法。在本机 rsync 3.4.4（protocol 32）之间，默认协商到 xxh128。和 3.2 以前的旧端通信时回落到 MD5（ $\text{protocol} \geq 30$ ）或 MD4（更老）。强校验具体怎么协商、版本边界在哪，是 [协议那篇](#) 的内容；这里只需要知道：它是一个足够强的哈希，用来在弱校验筛过之后给出可信的“相等”判断。

为什么两级就够？弱校验廉价但碰撞率高，强校验可信但贵。把弱校验当粗筛，发送端对每个偏移平均只付出一次哈希查表；只有极少数偏移能过弱校验这一关，才去付强校验的成本。两者叠起来，既快又可信。

下面这张图把单个偏移上的判定路径收拢成一条线：



命中后，发送端不再逐字节挪，而是直接把偏移跳过整个匹配块的长度，从块尾重新起算弱校验继续扫描——匹配到的整块都不再用再试探。

四、块大小与签名开销：sqrt 启发式

块长 L 是这套算法里唯一需要权衡的旋钮。块太大，一处小改动会让整块失配，复用率下降，传输量上升；块太小，块数变多，签名本身（每块一个弱校验加一段强校验）就要占掉可观的带宽，甚至盖过差异传输省下的量。

rsync 不让用户操心，默认按文件长度走一个平方根启发式。`generator.c` 的 `sum_sizes_sqrt()`：

```
/* generator.c, rsync 3.4.4, sum_sizes_sqrt() (节选) */
if (block_size) /* 用户用 -B 显式指定 */
    blength = block_size;
else if (len <= BLOCK_SIZE * BLOCK_SIZE) /* BLOCK_SIZE = 700 */
    blength = BLOCK_SIZE;
else {
    int32 max_blength = protocol_version < 30 ? OLD_MAX_BLOCK_SIZE : MAX_BLOCK_SIZE;
    /* ...取约等于 sqrt(len)、向 8 的倍数取整、并截到 max_blength... */
}
```

翻成结论：

- 文件不超过 $700 \times 700 = 490000$ 字节（约 480 KB）时，块长固定 700 字节。
- 更大的文件，块长约取 $\sqrt{\text{len}}$ ，向 8 的倍数取整。块数因此也大致是 $\sqrt{\text{len}}$ 量级，让“块数”和“块长”都随文件增长而增长，避免任何一侧爆掉。
- 块长有上限：protocol ≥ 30 时 `MAX_BLOCK_SIZE` 是 $2^{17} = 128 \text{ KB}$ （旧协议的 `OLD_MAX_BLOCK_SIZE` 是 2^{29} ）。

强校验的存储长度也不是定值。同一份 `sum_sizes_sqrt()` 还会算出每块强校验要保留多少字节 `s2length`：

```
/* generator.c, sum_sizes_sqrt(), protocol >= 27 的常规分支 */
int32 c;
int b = BLOCKSUM_BIAS; /* = 10 */
```



```

for (l = len; l >>= 1; b += 2) {} /* b += 2*floor(log2(len)) */
for (c = blength; (c >>= 1) && b; b--) {} /* b -= floor(log2(blength)) */
/* add a bit, subtract rollsum, round up. */
s2length = (b + 1 - 32 + 7) / 8; /* 减去 32 位弱校验已提供的强度 */
s2length = MAX(s2length, csum_length); /* 下限 */
s2length = MIN(s2length, MIN(SUM_LENGTH, xfer_sum_len)); /* 上限: 最多 16 字节, 且不超过协商出的摘要长度 */

```

逻辑是：文件越大、块越多，两个不同块的强校验偶然相同（碰撞）的概率必须压得越低，所以 `s2length` 随 $\log(\text{len})$ 增大；而 32 位弱校验本身已经提供了一部分区分度，可以从需要的强校验位数里扣掉。最终 `s2length` 被夹在一个下限和上限（最多 16 字节，且不超过协商出的摘要长度）之间。每块实际传输的强校验只是完整摘要的前 `s2length` 字节——签名够用就行，不浪费带宽。

这样，签名总开销 $\approx$ 块数 $\times$ (4 字节弱校验 + `s2length`)。在 sqrt 启发式下块数是 $\sqrt{\text{len}}$ 量级，签名相对文件本身小得多，又随文件增长保持碰撞概率可控。

五、增量编码与重建

发送端扫完 *A*，得到的是一串交织的指令：要么“接收端的第 *i* 个旧块”，要么“这一段是字面数据，照单收下”。`match.c` 的 `matched()` 把它们写进流里：匹配命中时，先把上次匹配点到本次命中之间那段没匹配上的字节作为字面数据发出，再发一个指向块号的 token；没有匹配、只是攒够了字面数据时，也通过同一个函数把字面数据冲刷出去。

```

/* match.c, matched() (节选) : i>=0 是匹配的块号, i<0 表示只有字面数据 */
static void matched(int f, struct sum_struct *s, struct map_struct *buf,
                   OFF_T offset, int32 i)
{
    int32 n = (int32)(offset - last_match); /* 本次命中前累积的字面数据长度 */
    send_token(f, i, buf, last_match, n, i < 0 ? 0 : s->sums[i].len);
    /* ...更新统计: i>=0 时计入 matched_data... */
    if (i >= 0)
        last_match = offset + s->sums[i].len; /* 跳到匹配块之后 */
    else
        last_match = offset;
}

```

接收端的重建因此非常直接：顺着这串指令走，遇到字面数据就写进新文件，遇到块引用就从自己本地的基准文件 *B* 里把对应的旧块拷过来。整个过程里，匹配上的块从不

经过网络——它本来就在接收端手上，网络上只走没匹配上的字面数据加上很短的 token。这正是“只传差异”的字面含义。

匹配块不经过网络带来一个推论：**rsync** 省的是带宽，不省读盘。接收端要把复用块从 *B* 读出来重写进新文件，发送端要把整个 *A* 读一遍来扫描和算校验。差异同步用两端的 CPU 和磁盘 I/O，换网络传输量。

为了防止匹配链路里出错（比如强校验在某个块上恰好撞了，或者传输位翻转），发送端在扫描 *A* 的同时，用 `sum_init` / `sum_update` / `sum_end` 顺手算出整个新文件的一个校验和，跟在 token 流后面发给接收端。接收端拼完后比对：对不上就重来一轮。块级的弱+强校验负责定位差异，这个文件级校验负责兜底正确性。

六、这套算法的适用边界

rsync 算法的成立条件很具体：新旧文件之间存在大量可对齐的相同片段。满足时，它在“不知道对方内容”的前提下，用 $O(|A|)$ 的扫描加少量强校验，把传输量压到接近真实差异。

反过来，几个场景它并不划算：

- 文件几乎全变（重新生成、压缩包重打、加密文件），旧块基本匹配不上，差异同步白白多算一遍校验，还不如直接传整文件。rsync 对此有 `--whole-file` 开关。
- 海量小文件，每个文件的差异算法都省不下多少，瓶颈转移到元数据和每文件握手，块级算法不是重点。
- 带宽极充裕而两端 CPU/磁盘紧张时，“用计算换带宽”这笔交易可能不成立。

这些“何时反而更慢”的判断需要实测撑腰，连同三进程模型、wire protocol、版本边界和 `--stats` 数据解读，都在 [协议、进程模型与实测](#) 那篇展开。本篇只交付一件事：在源端看不到目标文件的约束下，差异是怎么被一个 $O(1)$ 滚动的弱校验加一个可信的强校验，两级匹配算出来的。

参考资料

源码 (rsync 3.4.4) :

- `checksum.c` : `get_checksum1()` (弱校验)、`get_checksum2()` (强校验分派)、`csum_len_for_type()`。
- `match.c` : `build_hash_table()`、`hash_search()`、`matched()`、`match_sums()`，以及 `false_alarms` 计数。
- `generator.c` : `sum_sizes_sqrt()` (块长与 `s2length` 启发式)。
- `rsync.h` : `BLOCK_SIZE`、`MAX_BLOCK_SIZE`、`SUM_LENGTH`、`BLOCKSUM_BIAS`、`CHAR_OFFSET` 等常量。

规范与论文:

- Andrew Tridgell, Paul Mackerras. *The rsync algorithm*. Technical Report TR-CS-96-05, 1996.
- Andrew Tridgell. *Efficient Algorithms for Sorting and Synchronization*. PhD thesis, 1999, 第 3 章。